Equipment Monitoring & Predictive Maintenance System

A Full-Stack Platform for Real-Time Industrial Health Tracking

Subin Saju Adithya Chand A

UVJ Technologies

September 1, 2026

Introduction

Project Name

Equipment Monitoring & Predictive Maintenance System

Developed by:

- Subin Saju
- Adithya Chand A

Organization: UVJ Technologies

Purpose

- Monitor industrial equipment health in real time
- Detect anomalies before failures occur
- Enable data-driven, predictive maintenance
- Provide role-based access for plant staff

About the Project

What it does

- Continuously ingests telemetry (temperature, pressure, flow rate, runtime hours, error count) from equipment
- Sends each reading to a machine learning service that scores anomaly severity
- Classifies equipment risk as **LOW / MEDIUM / HIGH / CRITICAL**
- Lets maintenance engineers log maintenance actions against equipment
- Secures every action behind JWT authentication and role-based access control

Use case

- Factories and plants running rotating/industrial machinery that needs continuous condition monitoring
- Reduces unplanned downtime by flagging degrading equipment *before* it fails
- Gives operators, engineers, and administrators a single shared source of truth

Technology Stack

Backend

- ASP.NET Core 8 Web API
- Entity Framework Core
- SQL Server
- JWT-based authentication
- PBKDF2 password hashing

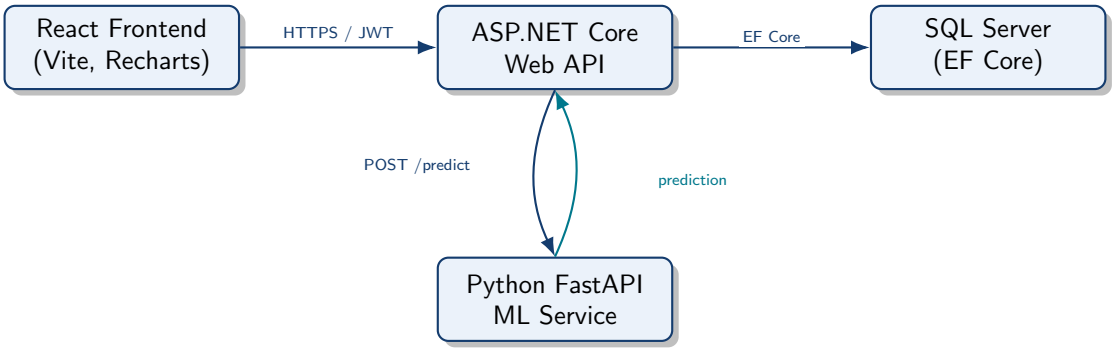
Frontend & ML

- React (Vite)
- Recharts (data visualization)
- Python + FastAPI

Architecture style

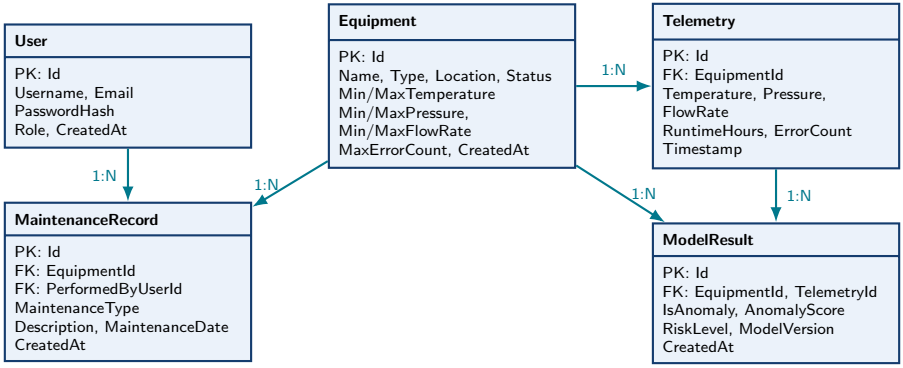
Three-tier architecture: React SPA frontend → ASP.NET Core REST API → SQL Server, with a decoupled Python FastAPI microservice for ML inference.

High-Level Architecture

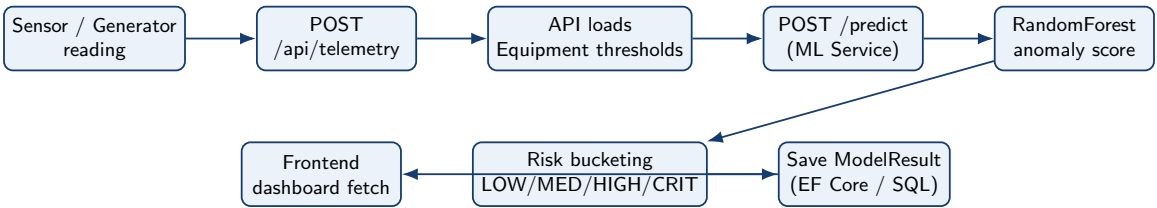


The React SPA talks only to the ASP.NET Core API. The API is the sole caller of the ML microservice and persists every prediction back to SQL Server.

Entity–Relationship Diagram



API Call Workflow: Telemetry to Prediction



Core REST Endpoints

Endpoint	Purpose	Allowed Roles
POST /api/auth/login	Authenticate, issue JWT	Everyone
POST /api/auth/register	Create a new user	Admin
GET /api/equipment	List / view equipment	Admin, Operator, Engineer
POST/PUT/DELETE /api/equipment	Create, update, remove equipment	Admin
GET /api/telemetry/equipment/{id}	View telemetry history	Admin, Operator, Engineer
GET /api/modelresult/faulty	List flagged / faulty equipment	Admin, Operator, Engineer
GET /api/maintenance	View maintenance records	Admin, Operator, Engineer
POST /api/maintenance	Log a maintenance action	Maintenance Engineer

User Roles & Permissions

Administrator

- Registers new users
- Creates, edits, deletes equipment
- Configures threshold profiles
- Full visibility across the system

Maintenance Engineer

- Views equipment & telemetry
- Reviews anomaly / risk results
- Logs maintenance records
- Closes the loop on flagged equipment

Operator

- Monitors live equipment status
- Views telemetry & risk dashboards
- Views maintenance history
- Read-focused, day-to-day watch role

The ML Model

Goal: replace hardcoded if/else threshold checks with a learned, continuous anomaly severity score.

- **Algorithm:** `RandomForestRegressor` (scikit-learn)
  300 trees, max depth 12, min samples per leaf 5
- **Inputs (12 features):** temperature, pressure, flowRate, runtimeHours, errorCount, and each equipment's min/max temperature, pressure, flow rate, and max error count
- **Output:** a continuous anomaly score in $[0, 1]$
- **Training data:** synthetically generated telemetry (20,000 samples) with a severity label based on how far each reading deviates from its allowed threshold band
- **Served via:** a standalone FastAPI microservice exposing a single `POST /predict` endpoint, called by the ASP.NET Core backend

From Score to Risk Level

The API stores every prediction as a `ModelResult` row linked to the `Equipment` and `Telemetry` that produced it.

Frontend: React Dashboard

- Built with **React** + Vite
- **Recharts** for live telemetry and risk visualizations
- JWT stored client-side; role claim decoded to drive UI visibility
- Pages map directly to the API surface:
  - Equipment list & detail views
  - Telemetry & model-result / risk dashboards
  - Maintenance record log
  - Admin-only user management

Summary

- End-to-end predictive maintenance platform: React $\rightarrow$ ASP.NET Core $\rightarrow$ SQL Server, with a Python ML microservice
- Role-based access for Admin, Maintenance Engineer, and Operator
- Random Forest model turns raw telemetry into an actionable risk level
- Secure, JWT-authenticated API with a clean, testable service/repository layout

Thank You